

INTERVIEW PREPARATION GUIDE

Frontend Developer WeMakeScholars

React · Next.js · Performance & SEO · Machine Coding · HR
Prepared for Sujith Mondithoka · Interview: 10:00 AM

How to use this guide. Work the sections in order. The plan on the next page is timed — if you fall behind, drop from the bottom, never from the top.

● **Red dot** = high priority, almost certainly asked. ●● = your differentiator. ● **Amber** = worth knowing.

✓ = self-check before moving on. ⚠ = common trap or something to verify.

Contents

Plan	WeMakeScholars — Frontend Developer Interview Prep	3
01	JavaScript Core	7
02	React (Highest priority)	14
03	Next.js (Your differentiator)	21
04	Performance & SEO	27
05	HTML, CSS & Responsive Design	31
06	REST API Integration	34
07	Machine Coding Round	38
08	HR, Behavioural & the Offer Conversation	46
09	Pitching Your Projects	51
10	Final Cheat Sheet — TOMORROW MORNING ONLY	54

WeMakeScholars — Frontend Developer Interview Prep

Your recruiter's war-room plan · Interview: tomorrow 10:00 AM

How to use this folder: Read this file first (10 min). Then work the numbered files in order. Every file follows the same 3-part structure as your Java revision plan: **Why it exists** → **Where it's useful** → **Questions & model answers**. Anything marked ● is *highly likely to be asked* — do not skip it.

1. Recruiter's read on this JD

I've placed dozens of candidates into orgs like this one. Here's what the JD is actually telling you, line by line:

What the JD says	What it really means for your interview
"React and Next.js"	Next.js is not optional garnish. They will ask rendering strategies (SSR/SSG/ISR/CSR) and why. File 03 is your highest-ROI file.
"Making Things Fast" (<i>highlighted in their own doc</i>)	The person who wrote this JD cares about it. Core Web Vitals, image optimization, bundle size, lazy loading. File 04.
"look great on every device"	Responsive CSS, Flexbox vs Grid, mobile-first. File 05.
"perform well on search engines"	SEO is a <i>revenue</i> function here — they acquire students via Google search. Metadata, semantic HTML, SSR-for-crawlers. File 04.
"plug REST APIs and data into the frontend"	fetch/axios, loading & error states, race conditions, auth headers. File 06.
"reusable components, readable, maintainable code"	They will ask you to justify a component API, and probably do a small machine-coding task. File 07.
"Partnering with designers and product"	Behavioural round. Have stories ready. File 08.
18-month commitment, 2-month notice, WFO, 1st & 3rd Sat working	These <i>will</i> be asked as filter questions. Scripted answers in File 08.

The single biggest differentiator in this interview: most candidates for a 4–6 LPA frontend role can *use* React. Very few can explain **why Next.js exists** and **what problem SSR solves for an SEO-driven business**. WeMakeScholars lives or dies on organic search traffic for education-loan keywords. If you connect your technical answers back to *"this helps your pages rank and load fast for students on slow mobile networks"*, you will sound like an engineer, not a bootcamp grad. Say it once in the interview. Once. It lands.

2. Likely interview structure (my prediction)

For a company of ~250 people hiring at 4-6 LPA, expect **2-3 rounds**, often compressed into one day:

1. **Technical screening (45-60 min)** — JS fundamentals, React hooks, "explain your project", CSS/responsive. Files 01, 02, 05.
2. **Machine coding / live task (45-60 min)** — build a small component: search+filter list, a form with validation, a counter/todo/accordion, or an API fetch with loading & error states. File 07.
3. **Tech lead / manager round (30 min)** — Next.js, performance, SEO, how you work with backend & design, code-quality opinions. Files 03, 04, 06, 08.
4. **HR (20 min)** — commitment, notice period, salary, relocation, working Saturdays. File 08.

3. THE SCHEDULE — from now until 10:00 AM tomorrow

You said you have until 10 AM tomorrow. Time is the scarce resource, so this plan is **ruthlessly prioritised**. If you fall behind, drop from the bottom, never from the top.

TODAY — Afternoon / Evening

Time	Block	File	Goal
T+0:00 → 0:15	Read this file	README.md	Know the battle plan
T+0:15 → 1:30	React core ●	02-react.md	Hooks, state, keys, lifecycle, re-renders
T+1:30 → 2:45	Next.js ●●	03-nextjs.md	SSR/SSG/ISR/CSR + routing + why it exists
T+2:45 → 3:00	Break — walk, no phone	—	Memory consolidates during breaks
T+3:00 → 4:00	JavaScript core ●	01-javascript.md	Closures, this , async, event loop
T+4:00 → 5:00	Performance + SEO ●	04-performance-seo.md	Core Web Vitals — their highlighted line
T+5:00 → 5:45	Dinner. Actually eat.	—	—
T+5:45 → 6:45	Machine coding ●	07-machine-coding.md	Type out 2 problems yourself. Don't read — <i>build</i> .
T+6:45 → 7:30	Project story ●	09-your-projects.md	Script your 2-min project pitch out loud

Time	Block	File	Goal
T+7:30 → 8:15	HR + behavioural	08-hr-and-behavioural.md	Commitment, notice, salary — say them aloud
T+8:15 → 8:45	REST APIs	06-rest-api-integration.md	Skim + the 4 code patterns
By 11:30 PM	SLEEP.	—	Non-negotiable. Tired > underprepared.

Cramming past midnight is a net negative. A sleep-deprived candidate blanks on `useEffect` dependency arrays they knew perfectly at 1 AM. I have watched it happen. Go to bed.

TOMORROW — Morning

Time	Block	File
07:00 - 07:20	Wake, shower, coffee, no phone doomscroll	—
07:20 - 08:00	Cheat sheet only — nothing new	10-final-cheatsheet.md
08:00 - 08:30	HTML/CSS skim	05-html-css-responsive.md
08:30 - 09:00	Say your project pitch + intro out loud, twice	09-your-projects.md
09:00 - 09:20	Re-read your questions for them	08-hr-and-behavioural.md \$6
09:20 - 09:40	Setup check: laptop, charger, network, ID, resume copies, water	—
09:40 - 09:55	Close every file. Breathe. Sit up straight.	—
10:00	Go.	—

Rule for tomorrow morning: learn NOTHING new. New material after 07:00 only displaces what's already consolidated and spikes your anxiety. Revision only.

4. Files in this folder

File	Topic	Priority
01-javascript.md	JS fundamentals — closures, <code>this</code> , event loop, async, ES6	● High

File	Topic	Priority
02-react.md	React — hooks, state, re-renders, keys, context, patterns	● Highest
03-nextjs.md	Next.js — SSR/SSG/ISR, routing, App Router, why it exists	● Highest
04-performance-seo.md	Core Web Vitals, optimization, SEO — <i>their highlighted line</i>	● High
05-html-css-responsive.md	Semantic HTML, Flexbox/Grid, responsive, accessibility	● Medium
06-rest-api-integration.md	fetch/axios, loading & error states, auth, race conditions	● Medium
07-machine-coding.md	Live-coding problems with full solutions	● High
08-hr-and-behavioural.md	HR filters, salary, commitment + questions to ask them	● High
09-your-projects.md	How to pitch <i>your</i> GitHub projects	● High
10-final-cheatsheet.md	One-page, tomorrow-morning-only revision	● Read last

5. Three rules for the room

1. **Never say "I don't know" and stop.** Say: *"I haven't used that in production, but my understanding is X — is that the direction you mean?"*
Interviewers hire people who reason out loud, not people who recite.
2. **Think out loud in the coding round.** Silence reads as being stuck. Narrate: *"I'll hold the input in state, debounce it, then filter the list..."*
3. **Tie answers to their business once.** *"Since students find you through Google, SSR matters here more than in a dashboard app."* That one sentence makes you memorable in a stack of 40 CVs.

6. Note on your resume

Your resume PDF lives on your local Windows machine (`C:\Users\sujit\Downloads\...`), which this session can't read. So `09-your-projects.md` is built from your **public GitHub repositories** instead. Open that file, check the project details match what's on your resume, and correct anything that's off — the interviewer will have the PDF in front of them, so the two must agree.

01 · JavaScript Core

Budget: 60 minutes · Every frontend interview opens here. If you're shaky on `this`, closures, or async, the rest of the interview never happens.

Where JavaScript fits (the "why" answer)

"HTML is the structure, CSS is the presentation, JavaScript is the behaviour. It's the only language browsers run natively, so every interaction — form validation, fetching loan data without a page reload, filtering a bank list — goes through it. React and Next.js are both just JavaScript libraries; nothing in them works if the fundamentals don't."

Q1 • `var` VS `let` VS `const`

	<code>var</code>	<code>let</code>	<code>const</code>
Scope	function	block <code>{}</code>	block <code>{}</code>
Redeclare	yes	no	no
Reassign	yes	yes	no
Hoisting	hoisted, <code>undefined</code>	hoisted into TDZ (ReferenceError)	same as <code>let</code>

Model answer: "`var` is function-scoped and gets hoisted as `undefined`, which causes bugs — the classic one is `var` in a loop with `setTimeout` printing the final value every time. `let` and `const` are block-scoped and sit in the Temporal Dead Zone until declared, so accessing them early throws instead of silently giving `undefined`. I default to `const`, use `let` when I must reassign, and never use `var`."

Follow-up trap: "Is a `const` object immutable?"

→ "No. `const` only prevents **rebinding the variable**. `const user = {}` then `user.name = 'A'` is fine; `user = {}` throws. For real immutability you'd use `Object.freeze()`, or just copy with spread."

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
```

Q2 • What is a closure? Where have you used one?

Definition: A closure is a function that remembers the variables from the scope where it was **created**, even after that outer function has returned.

```
function counter() {  
  let count = 0;           // private  
  return () => ++count;     // closes over `count`  
}  
  
const inc = counter();  
inc(); // 1  
inc(); // 2 - `count` survived, but nothing else can touch it
```

Where it's useful — say this, it's the part that impresses:

- **Data privacy / module pattern** — `count` can't be modified from outside.
- **`useState` in React is literally a closure** — the hook holds your state in a closure inside React's fiber, and gives you back a getter and a setter.
- **Debounce / throttle** — the timer ID is held in a closure between calls.
- **Event handlers** that need to remember which item they belong to.

Trap: "stale closure in React" — a `useEffect` with `[]` deps captures the first render's state forever. Fix: add the dep, or use the functional updater

```
setCount(c => c + 1).
```

Q3 • Explain the event loop / why is JS single-threaded but async?

Model answer (draw it if there's a whiteboard):

"JavaScript has one call stack, so it runs one thing at a time. Anything slow — a network call, a timer — is handed off to the browser's Web APIs, not to JS. When that finishes, its callback is queued. The **event loop** checks: if the call stack is empty, it pushes the next callback on. **Microtasks (Promises, `queueMicrotask`) drain completely before the next macrotask (`setTimeout` , events)** — that's the ordering people get wrong."

```
console.log('1');  
setTimeout(() => console.log('2'), 0); // macrotask  
Promise.resolve().then(() => console.log('3')); // microtask  
console.log('4');  
// Output: 1 4 3 2
```

Why it matters practically: a long synchronous loop blocks rendering and freezes the UI — this is exactly what hurts INP (see file 04).

Q4 • Callback → Promise → async/await

- **Callback:** function passed in, called when done. Nesting them = *callback hell*.

- **Promise:** an object representing a future value. States: `pending` → `fulfilled` | `rejected`. Chainable with `.then()`, flattening the nesting.
- **async/await:** syntactic sugar over Promises. `await` pauses that function until the Promise settles — reads like sync code, errors with `try/catch`.

```
async function getLoans() {
  try {
    const res = await fetch('/api/loans');
    if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch does NOT throw on 404/500
    return await res.json();
  } catch (err) {
    console.error(err);
    throw err;
  }
}
```

• Guaranteed follow-up: "How do you fire multiple API calls in parallel?"

```
const [banks, offers] = await Promise.all([getBanks(), getOffers()]);
```

"Sequential `await` s run one after another — two 300 ms calls take 600 ms. `Promise.all` fires both and waits for the slowest, so ~300 ms. If one rejecting shouldn't kill the rest, I use `Promise.allSettled`."

Method	Behaviour
<code>Promise.all</code>	all succeed, or reject on first failure
<code>Promise.allSettled</code>	waits for all, returns status of each
<code>Promise.race</code>	first to settle (win <i>or</i> lose) — good for timeouts
<code>Promise.any</code>	first to succeed

Q5 • `this`, and how arrow functions differ

`this` is decided by **how a function is called**, not where it's defined:

Call style	<code>this</code> is
<code>obj.fn()</code>	<code>obj</code>
<code>fn()</code> (standalone)	<code>undefined</code> in strict mode / <code>window</code> otherwise
<code>new Fn()</code>	the new instance
<code>fn.call(x)</code> / <code>.apply(x)</code> / <code>.bind(x)</code>	<code>x</code>

Call style	this is
Arrow function	inherited from the enclosing scope — cannot be rebound

```
const user = {
  name: 'Sujith',
  regular() { console.log(this.name); }, // 'Sujith'
  arrow: () => console.log(this.name), // undefined - took `this` from module scope
};
```

Practical takeaway: "Arrow functions are why we stopped writing `this.handleClick = this.handleClick.bind(this)` in class components. In modern React with hooks, `this` barely comes up — but it still matters for `setTimeout(function(){...})` and object methods."

Q6 ES6+ features you actually use daily

```
const { name, email = 'n/a' } = user; // destructuring + default
const [first, ...rest] = list; // rest
const next = { ...user, verified: true }; // spread (shallow copy)
const label = `Loan for ${name}`; // template literal
const city = user?.address?.city ?? 'Unknown'; // optional chaining + nullish coalescing
```

- `??` vs `||`: "`||` falls back on *any* falsy value, so `0 || 10` gives 10 — wrong if 0 is a legitimate interest rate. `??` only falls back on `null` / `undefined`."
- **Spread is a shallow copy:** nested objects are still shared references. Deep copy: `structuredClone(obj)`.

Q7 `==` vs `===` · `null` vs `undefined`

- `===` compares value **and** type, no coercion. Always use it.
- `==` coerces: `'5' == 5` → true, `null == undefined` → true, `0 == ''` → true.
- `undefined` = declared but never assigned (JS's default). `null` = *you* deliberately set "no value".
- Falsy list, memorise: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else — including `[]` and `{}` — is truthy.

Q8 • map vs forEach vs filter vs reduce

Method	Returns	Use for
<code>forEach</code>	<code>undefined</code>	side effects only
<code>map</code>	new array, same length	transform — <i>the React list-rendering one</i>
<code>filter</code>	new array, subset	search / filter features
<code>find</code>	first matching element	look up one item
<code>reduce</code>	any single value	totals, grouping, flattening

```
const total = loans.reduce((sum, l) => sum + l.amount, 0);
const byBank = loans.reduce((acc, l) => {
  (acc[l.bank] ||= []).push(l);
  return acc;
}, {});
```

"They're all non-mutating except `forEach` 's side effects, which is why they fit React — React needs a *new* array reference to detect a change."

Q9 Event bubbling, capturing, delegation

- **Bubbling:** event fires on target, then travels up to ancestors (default).
- **Capturing:** top-down phase, opt in with `addEventListener(fn, true)`.
- **Delegation:** put **one** listener on the parent and read `e.target` — instead of 100 listeners on 100 rows. Fewer listeners, and it works for rows added later.
- `e.stopPropagation()` stops the travel; `e.preventDefault()` stops the browser's default (form submit, link navigation).

Q10 • Debounce vs throttle — write this from memory

- **Debounce:** run only after the user *stops* for N ms → **search-as-you-type**.
- **Throttle:** run at most once per N ms → **scroll / resize / infinite scroll**.

```
function debounce(fn, delay = 300) {
  let timer; // closure!
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

function throttle(fn, limit = 300) {
  let waiting = false;
  return (...args) => {
    if (waiting) return;
    fn(...args);
    waiting = true;
    setTimeout(() => (waiting = false), limit);
  };
}
```

Say the business reason: "Without debouncing, a 10-character search fires 10 API calls. Debounced, it fires one — that's 90% fewer requests hitting the backend."

Q11 Hoisting · Temporal Dead Zone

Declarations are moved to the top of their scope at compile time.

`function` declarations hoist fully (callable before definition); `var` hoists as `undefined`; `let` / `const` hoist but stay in the TDZ, so early access **throws** — which is a feature, not a bug: it surfaces the mistake.

Q12 Shallow vs deep copy

```
const shallow = { ...original }; // nested objects still shared
const deep = structuredClone(original); // modern, handles Dates/Maps
const deepJson = JSON.parse(JSON.stringify(original)); // loses Date, undefined, functions
```

● In React this is *the* bug behind "my state changed but the UI didn't re-render" — you mutated the existing object instead of creating a new reference.

Q13 localStorage vs sessionStorage vs cookies

	Size	Lifetime	Sent to server?
localStorage	~5-10 MB	until cleared	no

	Size	Lifetime	Sent to server?
sessionStorage	~5 MB	until tab closes	no
cookies	~4 KB	Expires / Max-Age	yes, every request

"Auth tokens belong in an **httpOnly cookie** — JS can't read it, so an XSS script can't steal it. localStorage is readable by any script on the page. For a fintech-adjacent product handling student loan data, that distinction matters."

⚠ **Next.js note:** `localStorage` doesn't exist on the server. Access it inside `useEffect` or guard with `typeof window !== 'undefined'`.

Q14 Quick-fire round (they'll rattle these off)

- **slice vs splice?** `slice` returns a copy, non-mutating. `splice` mutates.
- **null == undefined?** `true`. With `===`? `false`.
- **typeof null?** `'object'` — a famous JS bug kept for backwards compatibility.
- **Check an array?** `Array.isArray(x)`.
- **NaN === NaN?** `false`. Use `Number.isNaN(x)`.
- **Higher-order function?** Takes or returns a function — `map`, `debounce`.
- **Pure function?** Same input → same output, no side effects. React components should be pure with respect to props.
- **Sync vs async?** Sync blocks the stack; async hands off and continues.
- **fetch on a 404?** It **resolves** — you must check `res.ok` yourself. Axios throws automatically. This one catches people out constantly.

✓ Before you move on, you should be able to say out loud:

1. A closure, with the counter example, in under 30 seconds.
2. Why `1 4 3 2` is the output of the event-loop snippet.
3. Debounce, written from memory, no reference.

02 · React (Highest priority)

Budget: 75 minutes · This is the core of the role. Expect 15–20 minutes of pure React questioning plus a coding task built on it.

Where React fits (the "why" answer — lead with this)

"Before React, updating a page meant hand-writing DOM manipulation — find the node, change it, keep it in sync with your data. That gets unmaintainable fast. React flips it: you describe **what the UI should look like for a given state**, and React figures out the minimal DOM changes. It's declarative instead of imperative. On top of that, the component model means a `LoanCard` or `BankRow` is written once and reused across the whole site — which is what the JD means by 'reusable components'."

Q1 • What is the Virtual DOM? How does reconciliation work?

"The Virtual DOM is a lightweight JS object tree describing the real DOM. When state changes, React builds a new tree, **diffs** it against the previous one, and applies only the changed nodes to the real DOM. Real DOM operations are expensive — they trigger layout and paint — so batching the minimum set of them is the win."

Diffing heuristics (this is the senior-sounding detail):

1. Different element **type** → tear down the old subtree, build a new one.
2. Same type → keep the node, update only changed attributes.
3. Lists are matched by `key`.

"**Is the Virtual DOM faster than direct DOM manipulation?**" — a trick question.

"Not inherently — hand-optimised DOM code can beat it. The Virtual DOM buys *predictability and maintainability* at a very good performance level, and stops me from accidentally writing something much slower."

Q2 • Why do lists need a `key`? Why is index a bad key?

"`key` gives React a stable identity for each item across renders. Without it, React falls back to position, so inserting at the top makes it think every item changed."

The example that proves you actually understand it:

"With `key={index}`, if I have a list of inputs and delete the first item, React sees the item that *was* index 1 now at index 0 and just updates its props — so the typed value from the deleted row stays visible on the wrong row. With a stable `key={item.id}`, React knows that item is gone and unmounts it correctly."

Index is acceptable **only** when the list is static, never reordered, filtered, or added to.

Q3 • Props vs State

Props	State
Passed in from parent	Owned inside the component
Read-only (immutable to the child)	Mutable via the setter
Change → parent re-renders child	Change → this component re-renders
Like function arguments	Like a local variable that persists

Follow-up: "How does a child update the parent?"

→ "The parent passes a callback down as a prop; the child calls it. Data flows down, events flow up — that's one-way data flow. If a value is needed by two siblings, I **lift the state up** to their nearest common parent."

Q4 • `useState` — the three traps they test

```
const [count, setCount] = useState(0);
```

Trap 1 — state updates are asynchronous and batched.

```
setCount(count + 1);  
setCount(count + 1); // only +1 total – both read the same stale `count`  
setCount(c => c + 1);  
setCount(c => c + 1); // +2 – functional updater gets the latest value
```

"Use the functional form whenever the new state depends on the old."

Trap 2 — never mutate state.

```
items.push(newItem); setItems(items); // ❌ same reference, no re-render  
setItems([...items, newItem]); // <span class="tick">✓</span> new reference  
setUser({ ...user, verified: true }); // <span class="tick">✓</span>
```

"React compares with `Object.is`. Same reference means 'nothing changed'."

Trap 3 — lazy initial state. `useState(expensiveFn())` runs on *every* render;
`useState(() => expensiveFn())` runs once.

Q5 • `useEffect` — dependency array, cleanup, and when NOT to use it

```
useEffect(() => { /* runs after every render */ });
useEffect(() => { /* runs once on mount */ }, []);
useEffect(() => { /* runs when `id` changes */ }, [id]);
useEffect(() => {
  const t = setInterval(tick, 1000);
  return () => clearInterval(t); // cleanup: on unmount + before next run
}, [id]);
```

What effects are for: synchronising with something *outside* React — data fetching, subscriptions, timers, direct DOM/browser APIs, analytics.

• **"When should you NOT use `useEffect`?"** — a strong-candidate question: "For anything derivable from existing state. If I have `items` and a `query`, the filtered list is just `items.filter(...)` computed during render — putting it in state plus an effect adds an extra render and a whole class of sync bugs."

Cleanup matters: without `clearInterval` / `removeEventListener` / `abort`, you leak memory and get "can't update state on an unmounted component" warnings.

Why an effect runs twice in dev: React 18 StrictMode intentionally mounts→unmounts→remounts to surface missing cleanup. Development only.

Q6 • All the hooks — one line each

Hook	Purpose
<code>useState</code>	local state
<code>useEffect</code>	side effects / sync with outside systems
<code>useContext</code>	read context without prop drilling
<code>useRef</code>	mutable value that doesn't trigger re-render; DOM access
<code>useMemo</code>	cache an expensive value between renders
<code>useCallback</code>	cache a function reference between renders
<code>useReducer</code>	complex state with many transitions (Redux-lite)
<code>useLayoutEffect</code>	like <code>useEffect</code> but fires before paint (measure DOM)
Custom hook	any reusable <code>useXyz</code> function that calls other hooks

• **Rules of Hooks + why:** Only call hooks at the **top level** of a component or custom hook — never inside conditions, loops, or nested functions. "React tracks hooks by **call order**, not by name. A conditional hook shifts the order between renders and React hands you the wrong state."

Q7 • useMemo VS useCallback VS useRef

```
const filtered = useMemo(() => items.filter(i => i.bank === bank), [items, bank]); // caches a
VALUE
const handleAdd = useCallback((id) => setCart(c => [...c, id]), []); // caches a
FUNCTION
const inputRef = useRef(null); // persists, no
re-render
```

- `useCallback(fn, deps)` is `useMemo(() => fn, deps)`.
- They only pay off when the child is wrapped in `React.memo`, or when the value is a dependency of another hook, or the computation is genuinely expensive.

● "Should you wrap everything in `useMemo`?" → "No. Memoization isn't free — it costs memory and a dependency comparison every render. Premature memoization adds noise. I profile with React DevTools first, then memoize the hot path." *That answer alone separates you from most candidates at this level.*

`useRef` two uses: (1) grab a DOM node — `inputRef.current.focus()`; (2) hold a mutable value across renders without re-rendering — a timer ID, a previous value, an "is first render" flag.

Q8 • What causes a re-render? How do you stop unnecessary ones?

Causes: own state changed · props changed · parent re-rendered · context value changed. (Note: **a parent re-rendering re-renders all children by default**, even if their props are identical.)

Fixes, in the order I'd apply them:

1. `React.memo(Child)` — skip re-render if props are shallow-equal.
2. `useCallback` / `useMemo` on props passed into memoized children (otherwise a fresh function/object reference defeats `React.memo` every time).
3. **Move state down** — keep it in the smallest component that needs it.
4. **Lift content up** / `children` — pass an expensive subtree as `children` so it isn't recreated by the re-rendering parent.
5. Split contexts so an unrelated update doesn't re-render every consumer.

Q9 • Controlled vs uncontrolled components

```
// Controlled – React state is the single source of truth
<input value={email} onChange={e => setEmail(e.target.value)} />

// Uncontrolled – the DOM holds the value, you read it via ref
<input ref={emailRef} defaultValue="" />
```

"I default to controlled: it makes validation, conditional disabling of the submit button, and formatting straightforward. Uncontrolled is fine for simple forms or file inputs (`<input type="file">` is always uncontrolled), and it avoids a re-render per keystroke on very large forms."

Q10 Context API — and its limits

```
const AuthContext = createContext(null);
<AuthContext.Provider value={{ user, login }}>{children}</AuthContext.Provider>
const { user } = useContext(AuthContext);
```

Solves: prop drilling — passing `user` through five layers that don't use it.

Good for: theme, auth/current user, language, cart.

● **Limit:** "Context isn't a state manager and it isn't optimised for frequent updates — **every consumer re-renders when the value changes**. If the value is an object literal in the Provider, that's a new reference every render, so everything re-renders. I memoize the value, and split high-frequency state into its own context or reach for Redux Toolkit / Zustand when the state graph gets big."

Q11 Class vs Function components / lifecycle mapping

Class	Hook equivalent
<code>componentDidMount</code>	<code>useEffect(fn, [])</code>
<code>componentDidUpdate</code>	<code>useEffect(fn, [dep])</code>
<code>componentWillUnmount</code>	the cleanup <code>return</code> inside <code>useEffect</code>
<code>this.state</code> / <code>setState</code>	<code>useState</code> / <code>useReducer</code>

"Function components with hooks are the standard now — less boilerplate, no `this`, and logic is reusable through custom hooks instead of HOCs and render props. I can read classes in legacy code, but I write functions."

Q12 Custom hooks — *have one ready, it's a green flag*

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

// usage: const q = useDebounce(query); useEffect(() => { search(q) }, [q]);
```

"A custom hook is just a function starting with `use` that calls other hooks. It shares **stateful logic**, not state — two components using `useDebounce` get independent state. I've used `useFetch`, `useDebounce`, and `useLocalStorage`."

Q13 Error boundaries · Suspense · lazy

```
const Dashboard = React.lazy(() => import('./Dashboard'));
<Suspense fallback={<Spinner />}><Dashboard /></Suspense>
```

- `React.lazy` + `Suspense` → **code splitting**: that route's JS downloads only when needed, shrinking the initial bundle (ties directly to file 04).
- **Error boundary** = a class component with `componentDidCatch` / `getDerivedStateFromError` that renders a fallback instead of a white screen. "Hooks can't do this yet — it's the one place I still write a class, or use `react-error-boundary`."

Q14 State management — what would you pick and why?

"Local `useState` first. Shared-but-simple → Context. Server data → **React Query** / **SWR**, because most 'global state' is actually cached server state and those libraries give you caching, refetching, and loading/error states for free. Genuinely complex client state → Redux Toolkit or Zustand. I wouldn't put Redux in a project this size just to hold a user object — that's ceremony without value."

Q15 Quick-fire

- **Fragment?** `<>...</>` — group children without an extra DOM wrapper node.
- **JSX?** Syntax sugar compiled to `React.createElement()`. Browsers never see JSX.
- **Why `className`?** `class` is a reserved word in JS.
- **Prop drilling?** Threading props through components that don't use them → Context.

- **Composition over inheritance?** React has no component inheritance — you nest and pass `children`.
 - **Conditional render?** `{isLoading && <Spinner/>}` or a ternary. `△ {count && ...}` renders a literal `0` when count is 0 — use `count > 0 && .`
 - **Can `key` be a prop you read?** No, `key` is consumed by React itself.
 - **`React.memo` vs `useMemo`?** `memo` = component; `useMemo` = value.
-

✓ Self-check before moving on

1. Explain re-render causes and three ways to reduce them.
2. Write `useDebounce` from memory.
3. Say the "index as key" failure case out loud in 20 seconds.

03 · Next.js (Your differentiator)

Budget: 75 minutes · The JD names Next.js explicitly and their business runs on Google search traffic. **This is the file that wins you the offer.**

Q1 ●● Why Next.js instead of plain React? (*near-certain first question*)

The model answer — memorise the shape of this, not the words:

"Plain React with Vite or CRA ships a client-side rendered app: the browser downloads an almost-empty HTML file plus a JS bundle, then React builds the page. Two problems. First, the user stares at a blank screen until the JS parses — painful on a mid-range Android on 4G. Second, the HTML a crawler first sees is essentially empty.

Next.js renders the page on the server, so the browser gets real HTML immediately — fast First Contentful Paint and fully crawlable content. On top of that it gives me file-based routing, API routes, automatic code splitting, `next/image` optimisation, and built-in metadata for SEO. For a site like WeMakeScholars where students arrive from Google searching for education loans, server rendering isn't a nice-to-have — it's how the pages get found."

If they push: "But Google executes JavaScript now."

→ "It does, but rendering is queued as a second pass, so indexing is slower and less reliable — and other crawlers (social preview bots, Bing, LinkedIn) are much worse at it. Server-rendered HTML removes the risk entirely. It also helps real users on slow devices, which is the bigger win."

Q2 ●● CSR vs SSR vs SSG vs ISR — *the highest-value table in this folder*

	When HTML is built	Best for	Next.js (App Router)	Pages Router
CSR	in the browser, after JS loads	dashboards, logged-in areas, anything private	' <code>use client</code> ' + <code>useEffect</code> fetch	same
SSR	on the server, on every request	personalised or always-fresh pages	<code>fetch(url, { cache: 'no-store' })</code>	<code>getServerSideProps</code>
SSG	at build time , once	blog, marketing pages, static content	default <code>fetch</code> (cached)	<code>getStaticProps</code>

	When HTML is built	Best for	Next.js (App Router)	Pages Router
ISR	at build, then regenerated in the background every N seconds	content that changes occasionally — <i>bank interest rates, loan listings</i>	<code>fetch(url, { next: { revalidate: 60 } })</code>	<code>getStaticProps</code> + <code>revalidate</code>

How to pick — say it as a decision rule:

- Content same for everyone and rarely changes → **SSG** (fastest possible, served from CDN).
- Same for everyone but updates periodically → **ISR** (SSG speed + freshness, no rebuild).
- Different per request / needs cookies or auth → **SSR**.
- Behind a login and SEO-irrelevant → **CSR** is fine.

● The follow-up that separates candidates: "Which would you use for WeMakeScholars' bank listing pages?"

"ISR. The list of partner banks and their rates is the same for every visitor, so per-request SSR wastes server work — but it does change, and a full rebuild for one rate change is impractical. `revalidate: 3600` gives me static-file speed and CDN caching, with content never more than an hour stale. A student's own application status page, on the other hand, is per-user, so that's SSR or CSR behind auth."

If you say only that in the entire interview, you've made the shortlist.

Q3 ● App Router vs Pages Router

	Pages Router (<code>/pages</code>)	App Router (<code>/app</code> , v13+)
Routing file	<code>pages/about.js</code>	<code>app/about/page.js</code>
Data fetching	<code>getServerSideProps</code> / <code>getStaticProps</code>	<code>async</code> Server Component with <code>await fetch</code>
Default component	client	Server Component
Layouts	<code>_app.js</code> , manual	nested <code>layout.js</code> , preserved across navigation
Loading UI	manual	<code>loading.js</code> (auto Suspense boundary)
Errors	<code>_error.js</code>	<code>error.js</code> per segment
Special files	—	<code>page</code> , <code>layout</code> , <code>loading</code> , <code>error</code> , <code>not-found</code> , <code>route</code>

"App Router is where Next is going and what new projects should use, but plenty of production apps are still on Pages Router — I'm comfortable reading and writing

both. **Be honest about which one you've actually built in** — say that, then show you understand the other."

Q4 ●● Server Components vs Client Components

```
// app/loans/page.js - Server Component by default
export default async function LoansPage() {
  const res = await fetch('https://api.example.com/loans', { next: { revalidate: 60 } });
  const loans = await res.json();
  return <LoanList loans={loans} />;
}

// components/SearchBar.jsx
'use client'; // opts into the browser bundle
import { useState } from 'react';
export default function SearchBar() {
  const [q, setQ] = useState('');
  return <input value={q} onChange={e => setQ(e.target.value)} />;
}
```

Server Component	Client Component
Runs on the server only	Runs on server (for HTML) and browser
Ships zero JS to the browser	Adds to the bundle
Can <code>await</code> directly, hit a DB, read secrets	Can use hooks, events, browser APIs
✗ no <code>useState</code> / <code>useEffect</code> / <code>onClick</code>	✓ all of it

The rule to state: "Keep components on the server by default and push `'use client'` down to the **leaves** — the interactive bits. If I put `'use client'` at the top of the page, everything under it goes to the browser and I've thrown away the benefit. A common pattern is a Server Component page that fetches data and renders a small client `<SearchBar>` inside it."

⚠ Environment variables: server-only by default. Only `NEXT_PUBLIC_*` reaches the browser — and anything with that prefix is **public**, never put a secret in one. Great question to nail if asked.

Q5 • Routing essentials

```
app/  
  layout.js           → root layout (html/body, shared nav)  
  page.js             → /  
  about/page.js       → /about  
  loans/[id]/page.js  → /loans/123      (dynamic segment)  
  blog/[...slug]/page.js → /blog/a/b/c    (catch-all)  
  (marketing)/page.js → route group – folder doesn't appear in the URL  
  loading.js  error.js  not-found.js  
  api/loans/route.js   → API endpoint (GET/POST exports)
```

```
// Navigation – client-side, no full page reload, prefetched in the viewport  
import Link from 'next/link';  
<Link href="/loans">Loans</Link>  
  
// Programmatic  
'use client';  
import { useRouter, useParams, useSearchParams } from 'next/navigation';  
const router = useRouter();  
router.push('/dashboard');
```

● **<Link> vs <a>** : " <a> triggers a full page reload — the whole app re-downloads and re-hydrates. <Link> does a client-side transition, only fetching the new route's payload, and prefetches links as they enter the viewport. But <Link> renders a real <a> underneath, so crawlers and middle-click still work."

⚠ App Router imports from `next/navigation` ; Pages Router uses `next/router` . Mixing them up is a classic slip — mention you know the difference.

Q6 • `next/image` — guaranteed if they care about performance

```
import Image from 'next/image';  
<Image src="/bank.png" alt="Partner bank" width={400} height={300}  
  priority /* only for above-the-fold / LCP image */ />
```

What it does for free: serves **WebP/AVIF** to browsers that support them, generates responsive `srcset` sizes, **lazy-loads** below-the-fold images, and **reserves the space** using width/height so the layout doesn't jump — that last one is your CLS score. `priority` disables lazy-loading for the hero image so LCP doesn't regress.

Same story for `next/font` : self-hosts the font at build time, so no round trip to Google Fonts and no flash of invisible text.

Q7 • SEO in Next.js — *their highlighted requirement*

```
// Static metadata – app/layout.js or any page.js
export const metadata = {
  title: 'Education Loans for Study Abroad | WeMakeScholars',
  description: 'Compare education loan offers from 15+ banks and NBFCs. Free.',
  openGraph: { title: '...', images: ['/og.png'] },
  alternates: { canonical: 'https://example.com/loans' },
};

// Dynamic, per-route
export async function generateMetadata({ params }) {
  const bank = await getBank(params.id);
  return { title: `${bank.name} Education Loan Interest Rates` };
}
```

Also mention, briefly: `sitemap.js` and `robots.js` file conventions, JSON-LD structured data injected via a `<script type="application/ld+json">` for rich results, and semantic headings — **one `<h1>` per page**.

Q8 What is hydration? What's a hydration error?

"The server sends HTML so the user sees content immediately, then React attaches event listeners to that existing markup in the browser — that's hydration. Until it finishes, the page looks ready but isn't interactive.

A **hydration mismatch** happens when the server HTML and the first client render differ — `new Date()`, `Math.random()`, `window / localStorage`, or browser-extension-injected markup. Fix: render that bit inside `useEffect` after mount, or use `next/dynamic` with `{ ssr: false }`."

Q9 API routes / Route Handlers

```
// app/api/leads/route.js
export async function POST(request) {
  const body = await request.json();
  // validate, call the real backend, keep API keys server-side
  return Response.json({ ok: true }, { status: 201 });
}
```

"Useful as a thin backend-for-frontend: hide an API key that must never reach the browser, reshape a backend response, or handle a form POST — without standing up a separate service."

Q10 Quick-fire

- **next/dynamic** ? Lazy-load a component; `{ ssr: false }` for browser-only libs (charts, maps).
- **Middleware**? Runs before a request completes — auth redirects, geo, A/B tests.
- **revalidatePath** / **revalidateTag** ? On-demand ISR invalidation after a content update.
- **Streaming**? With `loading.js` / `Suspense`, HTML streams in chunks so the shell paints while slow data resolves.
- **next build output**? Prints per-route whether it's Static ○, SSG ●, or Dynamic f — useful for verifying your rendering strategy.
- **Deploy**? Vercel is first-class; also `next build && next start` on any Node host, or `output: 'export'` for a fully static site.

✓ Self-check

1. Recite the CSR/SSR/SSG/ISR table and pick one for a bank-listing page **with a reason**.
2. Explain Server vs Client Components and the "push `'use client'` to the leaves" rule.
3. Explain hydration in three sentences.

If your Next.js experience is limited to a tutorial, say so honestly and then demonstrate the understanding above. "I've built with it at a project level rather than in production, but here's how I reason about rendering strategies..."
Recruiters forgive thin experience. Nobody forgives bluffing that unravels.

04 · Performance & SEO

Budget: 60 minutes · *"Making Things Fast"* was **highlighted in their own JD**.
Whoever wrote it cares. Treat this as a named requirement, not general knowledge.

Why this is a business requirement here, not a technical one

Say this once, early: *"WeMakeScholars acquires students through organic search. A slow page loses rankings and loses the student before the form even loads — most of them are on mid-range Android phones on 4G. So performance work here is directly lead generation, not polish."* This reframes you from coder to engineer.

Q1 ●● Core Web Vitals — know all three, with numbers

Metric	Measures	Good	Caused by	Fix
LCP — Largest Contentful Paint	when the main content appears	< 2.5s	huge hero images, slow server, render-blocking CSS/JS	optimise & preload the hero image (priority), SSR/SSG, CDN, compress
CLS — Cumulative Layout Shift	visual stability / jumping	< 0.1	images without dimensions, injected banners, late-loading fonts	always set width/height or aspect-ratio , reserve ad/banner space, font-display: swap
INP — Interaction to Next Paint	responsiveness to clicks/taps (<i>replaced FID in 2024</i>)	< 200ms	long JS tasks blocking the main thread	break up long tasks, debounce, memoize, ship less JS

Supporting: **TTFB** (server response, < 800 ms), **FCP** (first pixel of content).

If you remember one line: "LCP is *is it there*, CLS is *does it stay still*, INP is *does it respond*."

Q2 ● "The page is slow. How do you diagnose it?" — give a *process*

Never jump straight to fixes. Interviewers score the method:

- Measure first.** Lighthouse for a lab score, Chrome DevTools Performance + Network panels, and real-user data from **web-vitals** / Search Console if available. "I don't optimise what I haven't measured."
- Find the category.** Network (too much / too big / too slow)? Rendering (blocking resources)? JavaScript (long tasks)? Backend (high TTFB)?
- Fix the biggest item first** — usually images or bundle size.
- Re-measure** to prove the change worked, throttled to Slow 4G / 4× CPU.

Q3 • Your optimisation toolbox (grouped — recite by group, not randomly)

Images (usually the single biggest win)

- Modern formats — WebP/AVIF; `next/image` does this automatically.
- Correct dimensions + `srcset` — don't ship a 2000 px image into a 400 px slot.
- `loading="lazy"` below the fold; `priority` / `preload` for the LCP image.
- Always set width & height → kills CLS.

JavaScript / bundle

- **Code splitting** — `React.lazy` + `Suspense`, or `next/dynamic`, per route.
- **Tree shaking** — import what you use: `import { debounce } from 'lodash-es'`, not the whole library.
- Audit dependencies — moment.js → date-fns; do you need that carousel library?
- Defer third-party scripts (`next/script` with `strategy="lazyOnload"`) — analytics and chat widgets are frequently the worst offenders.

Rendering

- SSR/SSG so content paints before JS (see file 03).
- `React.memo` / `useMemo` / `useCallback` on measured hot paths.
- **Virtualise long lists** — `react-window` renders only visible rows; a 5,000-row table becomes 20 DOM nodes.
- Debounce search inputs, throttle scroll handlers.

Network / delivery

- CDN + HTTP caching headers; gzip/Brotli compression.
 - `preconnect` / `dns-prefetch` for critical third-party origins.
 - Self-host fonts (`next/font`), subset them, `font-display: swap`.
 - Avoid waterfalls — `Promise.all` parallel fetches instead of sequential awaits.
-

Q4 • Technical SEO checklist for a React/Next app

- **Server-render content that must rank.** CSR content is indexed slowly and unreliably.
- **Unique `<title>` and meta description per page** — `generateMetadata` in Next.
- **Semantic HTML:** one `<h1>`, ordered `<h2>/<h3>`, `<nav>`, `<main>`, `<article>`. Crawlers and screen readers both use structure.
- **`alt` text on every meaningful image** — accessibility *and* image search.
- **Canonical URLs** to prevent duplicate-content splits.
- `sitemap.xml` + `robots.txt` — Next has file conventions for both.
- **Structured data (JSON-LD)** — FAQ / Article / Organization schema for rich results. *A loan-comparison FAQ page is a textbook FAQ-schema use case — a great thing to volunteer.*
- **Open Graph / Twitter cards** for share previews.
- **Mobile-first & fast** — Google indexes the mobile version, and Core Web Vitals are a ranking signal.
- **Real `<a href>` links** for internal linking so crawlers can follow them. A `<div onClick={router.push}>` is invisible to a crawler — `<Link>` renders a proper anchor.

Q5 Accessibility (a11y) — the quick version

Worth a minute, and it overlaps with SEO:

- Semantic elements over `<div onClick>` — a real `<button>` is keyboard-focusable and announced correctly for free.
- Labels tied to inputs (`htmlFor` / `id`).
- Visible focus states; everything reachable by keyboard.
- Colour contrast $\geq 4.5:1$ for body text.
- `aria-label` only when there's no visible text (icon buttons); ARIA is a patch, not a substitute for semantics.
- Test: tab through the page; run the Lighthouse a11y audit.

Q6 Caching, briefly

- **Browser cache** via `Cache-Control` — hashed asset filenames let you cache JS/CSS for a year and bust on deploy.
- **CDN cache** — static pages served from an edge near the user.
- **Data cache** — Next's `fetch` cache / ISR `revalidate` ; React Query / SWR on the client.
- **Memoization** — cache within a single render tree.

Q7 Likely rapid-fire

- **Reduce initial bundle?** Code split by route, lazy-load below-the-fold components, tree-shake, drop heavy deps, analyse with `@next/bundle-analyzer` .
- **Lazy loading?** Defer loading a resource until it's needed/near the viewport.
- **CDN?** Geographically distributed servers caching static assets close to the user — lower latency, less origin load.
- **Critical rendering path?** HTML → DOM, CSS → CSSOM → Render Tree → Layout → Paint. CSS is render-blocking; `<script>` in `<head>` without `defer` is parser-blocking.
- **defer vs async ?** Both download in parallel. `async` executes the moment it lands (order not guaranteed); `defer` executes after HTML parsing, in order. Use `defer` for anything that touches the DOM.
- **Reflow vs repaint?** Reflow = geometry recalculated (expensive); repaint = pixels redrawn (cheaper). Animate `transform` / `opacity` , which skip both and run on the compositor — never animate `width` / `top` .
- **How would you measure improvement?** Lighthouse before/after + real-user CWV, on a throttled mobile profile.

✓ Self-check

1. Name LCP, CLS, INP with their thresholds and one fix each.

2. Give the four-step diagnosis process for "the page is slow".
3. Explain why CSR is a risk for SEO in one sentence.

05 · HTML, CSS & Responsive Design

Budget: 30 min tonight, skim tomorrow · "Translating UI/UX wireframes into clean, performant, responsive code" is the JD's opening line — this comes up in the coding round more than in questioning.

Q1 • Semantic HTML — why bother?

```
<header> <nav> <main> <section> <article> <aside> <footer> <figure> <button>
```

"Semantic tags describe *meaning*, not appearance. Three payoffs: screen readers can navigate by landmark, search engines understand page structure, and the markup is readable to the next developer. A `<div onClick>` is invisible to a keyboard user and to a crawler — a `<button>` is focusable, activates on Enter/Space, and is announced as a button, all for free."

Q2 • The box model

```
content → padding → border → margin
```

```
* { box-sizing: border-box; } /* width now includes padding + border */
```

"With the default `content-box`, adding padding makes the element wider than the width you set, which breaks grid layouts. `border-box` makes width mean what you expect. It's the first line of nearly every reset."

Q3 • Flexbox vs Grid — *when do you use which?*

- **Flexbox = one dimension** (a row **or** a column). Navbars, button groups, centring, card internals, space distribution.
- **Grid = two dimensions** (rows **and** columns at once). Page layouts, card galleries, dashboards.
- "They compose — a Grid page layout with Flexbox inside each card is the normal setup."

```
.nav { display: flex; justify-content: space-between; align-items: center; gap: 1rem; }  
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 1.5rem; }  
.center { display: grid; place-items: center; } /* centring, done */
```

• `auto-fit` + `minmax` is a responsive grid with no media queries at all — demo that line and explain it. It reads as experience.

Flex axis vocabulary: `justify-content` = main axis, `align-items` = cross axis.

```
flex: 1 = flex-grow:1 flex-shrink:1 flex-basis:0 .
```

Q4 • Responsive design — mobile-first

```
.card { padding: 1rem; } /* mobile base */
@media (min-width: 768px) { .card { padding: 2rem; } } /* tablet+ */
@media (min-width: 1024px) { .card { padding: 3rem; } } /* desktop+ */
```

"I write mobile-first with `min-width` queries: the base styles are the simplest case and each breakpoint adds complexity. `max-width` queries mean the phone — usually the weakest device — downloads and overrides desktop styles."

Essentials: `<meta name="viewport" content="width=device-width, initial-scale=1">` (without it, nothing responsive works), relative units (`rem` , `%` , `vw` , `ch`), `clamp(1rem, 2.5vw, 2rem)` for fluid type, `max-width: 100%` on images, and container queries when a component must adapt to *its container*, not the viewport.

Q5 Position values

`static` (default) · `relative` (offset from itself, keeps its space, creates a positioning context) · `absolute` (removed from flow, positioned against nearest positioned ancestor) · `fixed` (against the viewport — sticky headers) · `sticky` (relative until it hits a threshold, then fixed — needs `top` set).

Q6 Specificity & the cascade

inline (1000) > id (100) > class/attribute/pseudo-class (10) > element (1).

`!important` overrides everything — "a debugging smell; I'd rather fix the selector. Consistent low specificity, or a utility framework like Tailwind, avoids the arms race entirely."

Q7 Quick-fire

- `display: none` vs `visibility: hidden` vs `opacity: 0` ? Removed from layout / hidden but occupies space / visible to layout **and** still clickable.
- **Pseudo-class vs pseudo-element?** `:hover` (a state) vs `::before` (a generated sub-element).
- `rem` vs `em` ? `rem` = root font size (predictable); `em` = parent's (compounds when nested).
- **Centre a div?** `display: grid; place-items: center` — or flex with `justify-content` + `align-items` .
- **CSS variables?** `--brand: #0055ff;` / `var(--brand)` . Runtime-changeable — the standard way to do theming and dark mode.
- **Z-index not working?** It only applies to positioned elements, and it's scoped to the parent's **stacking context** — a parent with `transform` or `opacity < 1` traps children.

- **Tailwind opinion?** "Utility classes keep styles co-located with markup, no naming debates, and unused CSS is purged so the bundle stays small. Trade-off is noisy JSX — I pull repeated patterns into components or `@apply` ." (*Have an opinion; don't be dogmatic.*)
 - **SCSS?** Nesting, variables, mixins, partials — compiles to CSS. Less needed now that CSS has nesting and custom properties.
 - **CSS Modules?** `styles.card` — locally scoped class names, no global collisions. Next.js supports them out of the box.
-

✓ Self-check

1. Flexbox vs Grid, decided in one sentence.
2. Write the `auto-fit` / `minmax` responsive grid from memory.
3. Explain mobile-first and why `min-width` beats `max-width` .

06 • REST API Integration

Budget: 30 minutes • The JD: "Working closely with our backend team to plug REST APIs and data into the frontend smoothly." Expect this in the coding round — fetch, loading, error, empty.

Q1 REST basics they may check

Verb	Use	Idempotent?
GET	read	yes
POST	create	no
PUT	replace whole resource	yes
PATCH	partial update	no (usually)
DELETE	remove	yes

Status codes: 200 OK • 201 Created • 204 No Content • **400** bad request • **401** not authenticated • **403** authenticated but not allowed • **404** not found • **429** rate limited • **500** server error.
• 401 vs 403 is a common gotcha — "401 is 'who are you?', 403 is 'I know who you are and you still can't.'"

Q2 • fetch vs axios

fetch	axios
built in, no dependency	~13 KB library
doesn't reject on 4xx/5xx — check <code>res.ok</code>	rejects on error status automatically
manual <code>JSON.stringify</code> + <code>.json()</code>	auto JSON both ways
no built-in timeout (use <code>AbortSignal.timeout</code>)	<code>timeout</code> option
no interceptors	interceptors — attach auth token / handle 401 globally

"For a small app, `fetch` is enough. On a bigger codebase I like axios interceptors: one place to attach the auth header and one place to catch a 401 and redirect to login, instead of repeating it in every call."

Q3 ●● The four states — *say all four, most candidates say two*

"Every data-driven UI has four states: loading, error, empty, and success. Empty is the one people forget — 'no loans match your filters' should be a designed state, not a blank screen." That sentence is a hiring signal.

```
function LoanList() {
  const [loans, setLoans] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    (async () => {
      try {
        setLoading(true);
        setError(null);
        const res = await fetch('/api/loans', { signal: controller.signal });
        if (!res.ok) throw new Error(`Request failed: ${res.status}`);
        setLoans(await res.json());
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        setLoading(false);
      }
    })();

    return () => controller.abort(); // <span class="dot dot-hi"></span> cleanup: cancel on
    unmount
  }, []);

  if (loading) return <Spinner />;
  if (error) return <ErrorState message={error} onRetry={refetch} />;
  if (!loans.length) return <p>No loans match your filters.</p>;
  return <ul>{loans.map(l => <LoanCard key={l.id} loan={l} />)}</ul>;
}
```

Point out three things in this snippet unprompted:

1. `res.ok` — because `fetch` doesn't throw on 404/500.
2. `AbortController` in the cleanup — prevents the state update on an unmounted component and stops a **race condition** where a slow earlier request resolves after a fast later one and overwrites fresh data with stale.
3. `finally` — loading always ends, even on failure.

Q4 • Race conditions in search-as-you-type

"If the user types 'ax' then 'axis', two requests are in flight. If 'ax' resolves last, the UI shows the wrong results. Three fixes: abort the previous request with `AbortController` (my default), debounce so fewer requests are made at all, or tag each request and ignore stale responses. React Query handles this for me."

Q5 Auth on the frontend

- Token in an **httpOnly cookie** > `localStorage` (XSS-proof). Cookies need `credentials: 'include'` cross-origin.
 - Attach `Authorization: Bearer <token>` via an axios interceptor.
 - On **401** → clear session and redirect to login, in one interceptor.
 - Refresh-token flow: on 401, try refresh once, retry the original request, else log out.
 - **Never put secrets in frontend code.** In Next, anything `NEXT_PUBLIC_*` is visible in the browser bundle — proxy through a Route Handler instead.
-

Q6 CORS — you will be asked at least the definition

"CORS is a **browser** security rule: a page on origin A can't read a response from origin B unless B sends `Access-Control-Allow-Origin`. It's enforced by the browser, not the server, which is why the same request works in Postman. **It's fixed on the backend, not the frontend** — a dev proxy is a local workaround, not a fix. For non-simple requests the browser first sends an OPTIONS preflight."

Q7 React Query / SWR — worth mentioning

"Most 'global state' is really cached server state. React Query gives caching, background refetching, deduped requests, retries, pagination, and loading/error/stale flags in a few lines — replacing a lot of `useEffect` + `useState` boilerplate and the race-condition handling above."

```
const { data, isLoading, error } = useQuery({ queryKey: ['loans'], queryFn: getLoans });
```

Q8 Working with the backend team — *the JD's actual ask*

Answer as a collaborator, not just a coder:

"I ask for the contract early — endpoint, request shape, response shape, error shape — and agree on it before either side builds. If the API isn't ready, I mock the response so I'm not blocked and swap the URL later. I check pagination and error formats up front, because those are what break in integration. And when a

response shape doesn't fit the UI, I'd rather raise it than quietly write transformation code in five components."

✓ Self-check

1. Recite the four UI states.
2. Explain why `AbortController` is in the cleanup — the two reasons.
3. Explain CORS in two sentences, including whose job it is to fix.

07 · Machine Coding Round

Budget: 60 minutes — and TYPE these, don't read them.

Reading code creates the illusion of competence. Open StackBlitz/CodeSandbox and build problems 1 and 2 from scratch. That's the whole hour.

| How to run the round (the process is half the score)

1. **Clarify for 60 seconds.** "Should filtering be client-side or via API? Do you want it debounced? Should state persist on refresh?" *Candidates who clarify consistently outscore candidates who start typing immediately.*
2. **Say your plan out loud.** "State: `query`, `items`, `loading`. I'll debounce the input, derive the filtered list during render, then handle the empty state."
3. **Build the ugly working version first.** Working > pretty. You can style at the end.
4. **Narrate while typing.** Silence reads as stuck.
5. **Handle the edge cases out loud** — empty list, error, 0 results, long text. Even saying "I'd add an error state here if I had more time" scores.
6. **Refactor at the end** if time allows: extract a component, pull out a custom hook.

Their favourite problems for this level: search/filter list · todo CRUD · counter · accordion/tabs · star rating · form validation · fetch users and display · pagination · modal · countdown timer.

| Problem 1 ● — Search + filter list with debounce

The single most-asked machine-coding question in Indian frontend interviews.

```

import { useState, useEffect, useMemo } from 'react';

function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

export default function BankSearch({ banks }) {
  const [query, setQuery] = useState('');
  const [type, setType] = useState('all');
  const debouncedQuery = useDebounce(query);

  // derived state – NOT useState + useEffect
  const filtered = useMemo(() => {
    return banks.filter(b => {
      const matchesQuery = b.name.toLowerCase().includes(debouncedQuery.toLowerCase().trim());
      const matchesType = type === 'all' || b.type === type;
      return matchesQuery && matchesType;
    });
  }, [banks, debouncedQuery, type]);

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search banks..."
        aria-label="Search banks"
      />
      <select value={type} onChange={e => setType(e.target.value)}>
        <option value="all">All</option>
        <option value="public">Public</option>
        <option value="private">Private</option>
        <option value="nbfc">NBFC</option>
      </select>

      {filtered.length === 0
        ? <p>No banks match "{debouncedQuery}"</p>
        : <ul>{filtered.map(b => <li key={b.id}>{b.name} – {b.rate}%</li>)}</ul>}
    </div>
  );
}

```

Points to say aloud: filtered list is **derived during render**, not stored in state · debounce cuts API/compute churn · `key` is a stable id · empty state handled · `.trim()` and case-insensitive matching are the edge cases they check.

Problem 2 ● — Fetch + loading / error / empty / retry

```
import { useState, useEffect, useCallback } from 'react';

export default function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const load = useCallback(async (signal) => {
    try {
      setLoading(true);
      setError(null);
      const res = await fetch('https://jsonplaceholder.typicode.com/users', { signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setUsers(await res.json());
    } catch (e) {
      if (e.name !== 'AbortError') setError(e.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    const c = new AbortController();
    load(c.signal);
    return () => c.abort();
  }, [load]);

  if (loading) return <p>Loading...</p>;
  if (error) return (
    <div role="alert">
      <p>Couldn't load users: {error}</p>
      <button onClick={() => load()}>Retry</button>
    </div>
  );
  if (!users.length) return <p>No users found.</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

The retry button is the detail almost nobody adds. Add it.

Problem 3 — Todo list (CRUD + no mutation)

```
export default function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  const add = (e) => {
    e.preventDefault();
    const value = text.trim();
    if (!value) return; // edge case: empty input
    setTodos(t => [...t, { id: crypto.randomUUID(), text: value, done: false }]);
    setText('');
  };

  const toggle = (id) =>
    setTodos(t => t.map(todo => todo.id === id ? { ...todo, done: !todo.done } : todo));

  const remove = (id) => setTodos(t => t.filter(todo => todo.id !== id));

  return (
    <>
      <form onSubmit={add}>
        <input value={text} onChange={e => setText(e.target.value)} />
        <button type="submit">Add</button>
      </form>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <input type="checkbox" checked={todo.done} onChange={() => toggle(todo.id)} />
            <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>{todo.text}</span>
            <button onClick={() => remove(todo.id)}>x</button>
          </li>
        ))}
      </ul>
      <p>{todos.filter(t => !t.done).length} remaining</p>
    </>
  );
}
```

Highlights: `<form onSubmit>` so **Enter works** · functional updaters ·
map/filter never mutate · `crypto.randomUUID()` not array index · the remaining
count is derived, not stored.

Problem 4 — Form with validation

```
export default function LeadForm() {
  const [values, setValues] = useState({ name: '', email: '', phone: '' });
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});
  const [status, setStatus] = useState('idle'); // idle | submitting | success | error

  const validate = (v) => {
    const e = {};
    if (!v.name.trim()) e.name = 'Name is required';
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v.email)) e.email = 'Enter a valid email';
    if (!/^[6-9]\d{9}$/.test(v.phone)) e.phone = 'Enter a valid 10-digit mobile number';
    return e;
  };

  const handleChange = (e) => {
    const next = { ...values, [e.target.name]: e.target.value };
    setValues(next);
    if (touched[e.target.name]) setErrors(validate(next)); // re-validate only after blur
  };

  const handleBlur = (e) => {
    setTouched(t => ({ ...t, [e.target.name]: true }));
    setErrors(validate(values));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const found = validate(values);
    setErrors(found);
    setTouched({ name: true, email: true, phone: true });
    if (Object.keys(found).length) return;
    setStatus('submitting');
    try {
      await fetch('/api/leads', { method: 'POST', body: JSON.stringify(values) });
      setStatus('success');
    } catch { setStatus('error'); }
  };

  return (
    <form onSubmit={handleSubmit} noValidate>
      {[ 'name', 'email', 'phone' ].map(field => (
        <div key={field}>
          <label htmlFor={field}>{field}</label>
          <input id={field} name={field} value={values[field]}
            onChange={handleChange} onBlur={handleBlur}
            aria-invalid={!!errors[field]} />
          {touched[field] && errors[field] && <small role="alert">{errors[field]}</small>}
        </div>
      )]}
    </form>
  );
}
```

```

        </div>
      )})
      <button disabled={status === 'submitting'}>
        {status === 'submitting' ? 'Submitting...' : 'Submit'}
      </button>
      {status === 'success' && <p>Thanks – we'll call you shortly.</p>}
    </form>
  );
}

```

The senior touch: errors only show **after blur**, not while the user is still typing their first character; the submit button is disabled while in flight to prevent double submission; single `handleChange` keyed by `name`.

Problem 5 — Accordion (controlled, one open at a time)

```

function Accordion({ items }) {
  const [openId, setOpenId] = useState(null);
  return (
    <div>
      {items.map(item => {
        const isOpen = openId === item.id;
        return (
          <div key={item.id}>
            <button aria-expanded={isOpen}
              onClick={() => setOpenId(isOpen ? null : item.id)}>
              {item.title} {isOpen ? '-' : '+'}
            </button>
            {isOpen && <p>{item.body}</p>}
          </div>
        );
      })}
    </div>
  );
}

```

"One `openId` instead of a boolean per item makes 'only one open' impossible to get wrong. For multi-open I'd hold a `Set`. `aria-expanded` makes it accessible." —
and an FAQ accordion with JSON-LD is an SEO win, which you can mention.

Problem 6 — Star rating

```
function Rating({ value = 0, onChange, max = 5 }) {
  const [hover, setHover] = useState(0);
  return (
    <div onMouseLeave={() => setHover(0)}>
      {Array.from({ length: max }, (_, i) => i + 1).map(star => (
        <button key={star} type="button"
          aria-label={`Rate ${star} of ${max}`}
          onMouseEnter={() => setHover(star)}
          onClick={() => onChange(star)}>
          {star <= (hover || value) ? '★' : '☆'}
        </button>
      ))}
    </div>
  );
}
```

`hover || value` is the whole trick — hover preview falls back to the committed value.

Problem 7 — Pagination (client-side)

```
function Paginated({ items, perPage = 10 }) {
  const [page, setPage] = useState(1);
  const totalPages = Math.ceil(items.length / perPage);
  const slice = items.slice((page - 1) * perPage, page * perPage);

  useEffect(() => { setPage(1); }, [items]); // reset when the data set changes

  return (
    <>
      <ul>{slice.map(i => <li key={i.id}>{i.name}</li>)}</ul>
      <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>Prev</button>
      <span>Page {page} of {totalPages} | 1</span>
      <button disabled={page >= totalPages} onClick={() => setPage(p => p + 1)}>Next</button>
    </>
  );
}
```

Edge cases to name: disabled buttons at the boundaries, empty list, resetting to page 1 when filters change. (You've built `xpagination` before — say so.)

Problem 8 — Counter / timer (the `useRef` + cleanup check)

```
function Timer() {
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(false);
  const idRef = useRef(null);

  useEffect(() => {
    if (!running) return;
    idRef.current = setInterval(() => setSeconds(s => s + 1), 1000); // functional updater!
    return () => clearInterval(idRef.current); // cleanup!
  }, [running]);

  return (
    <>
      <h1>{seconds}s</h1>
      <button onClick={() => setRunning(r => !r)}>{running ? 'Pause' : 'Start'}</button>
      <button onClick={() => { setRunning(false); setSeconds(0); }}>Reset</button>
    </>
  );
}
```

They are checking exactly two things: `setSeconds(s => s + 1)` (not `seconds + 1`) and the `clearInterval` cleanup. Point both out.

If they ask for plain JS/DOM instead

```
document.querySelector('#list').addEventListener('click', (e) => { // delegation
  const btn = e.target.closest('button[data-id]');
  if (!btn) return;
  removeItem(btn.dataset.id);
});
```

✓ Tonight's deliverable

Build **Problem 1** and **Problem 2** from a blank file, no copy-paste. If you can do those two while talking, you can handle anything they throw at this level.

08 · HR, Behavioural & the Offer Conversation

Budget: 45 minutes — and say every answer OUT LOUD.

Reading these silently is worth about 20% of saying them. Your mouth needs the reps, not your eyes.

1. • "Tell me about yourself" — the first 90 seconds

This is the only answer worth memorising almost word-for-word, because it sets the tone for everything after. Structure: **Present → Path → Why here.**

"I'm a frontend developer working primarily with React and JavaScript. Most of my work has been building interfaces from designs and wiring them to REST APIs — I've built a multi-step form with validation, a paginated data table, an expense tracker, and several component-level UI builds, all on my GitHub.

What I've focused on recently is the part beyond 'it works' — how fast the page loads, how it behaves on a mid-range phone, and writing components other people can reuse.

That's why this role stood out. WeMakeScholars gets students through search, so the frontend has to be fast and crawlable — that's exactly the Next.js and performance work I want to be doing full-time, and on a product where the page loading properly actually decides whether a student gets a loan."

Rules: under 2 minutes · no life story from school onwards · end on *why them*.

Practise it twice out loud tonight and twice tomorrow morning. That's it.

2. • "Why WeMakeScholars?" — do the homework, it's 3 sentences

Use their own facts back at them:

"Three reasons. It's a 10-year-old, 250-person organisation, so it's stable but still building — I'd rather own real features than maintain someone else's legacy code. Second, the product is free for students and funded under Digital India, so the frontend genuinely decides whether someone finds and completes a loan application — the work has a visible outcome. Third, it's a search-driven product, which means Next.js, SSR and Core Web Vitals aren't optional here. That's the skill set I want to grow in."

Never say: "I need a job", "I want to learn", "the package is good".

Do mention you know they were inaugurated by Dr. Shashi Tharoor / recognised by Forbes / won the ET award — once, lightly. It shows you read.

3. • The filter questions — answer instantly and without hedging

These decide whether you move forward. Hesitation here reads as "will leave in six months", which is precisely what the 18-month clause exists to prevent.

"We need an 18-month commitment. Are you comfortable?"

"Yes. I'm looking for a place to build depth, not to switch in a year — 18 months is about the minimum time to actually own something end to end. I'd just like to understand how it's documented in the offer."

(Asking that follow-up is not a red flag — it's professional. Do read the clause before you sign: ask whether it's a written commitment or a bond with a financial penalty, and whether any amount is recoverable if you leave early. Ask it calmly at offer stage, not in round one.)

"What's your notice period?" → State it plainly and give a joining date.

"I can join in X days." *(Their own 2-month notice applies once you're employed there — nothing to answer now.)*

"Are you okay with work from office in Hyderabad?" → "Yes." Full stop. Don't open a negotiation you weren't asked to have.

"1st and 3rd Saturdays are working." → "That's fine — I saw it in the JD, and it makes sense given you work with banks."

"Timings are 10-11 AM login, 8 hours." → "Works for me."

⚠ If any of these are genuinely a problem for you, say so **now**, politely. Accepting and renegotiating later burns the relationship and your reference.

4. • Salary — the CTC is 4-6 LPA, so play the band

"What are your salary expectations?"

If you have prior experience / a current CTC:

"I'm looking in the range of ₹X to ₹Y. That said, I'm more focused on the role and the learning here than on a specific number — if the fit is right on both sides, I'm confident we can agree on something."

If you're early-career with no strong anchor:

"I'd like to understand the band for this role first. I saw the JD mentions 4 to 6 LPA — I'm comfortable within that range, and I'd expect to be placed based on how the technical rounds go."

Tactics:

- **Don't name a number below 4** — you'd be bidding against yourself under their own published floor.
- If you want the top of the band, **earn it in the technical round first**, then cite it: "Based on the Next.js and performance work we discussed, I'd be looking at the upper end."
- Ask **CTC vs in-hand** breakdown at offer stage — how much is fixed, variable, and what's deducted. A 6 LPA CTC and a 6 LPA fixed are different jobs.
- Never lie about your current CTC. Payslip verification is routine.

5. Behavioural questions — answer with STAR

Situation → Task → Action → Result. Keep it under 90 seconds and always land on a result.

"Tell me about a challenging bug."

"In my multi-step form, state was resetting when users moved between steps. (S/T) I'd been remounting the step components, so their local state was being destroyed. (A) I lifted the form state into the parent and passed values down, so each step became a controlled presentational component. (R) The bug went away and the code got simpler — adding a new step became a config change instead of new state logic. It taught me to ask *who owns this state* before writing it."

"A time you disagreed with someone."

"A design had a filter panel that pushed the content far down on mobile. I flagged that it would hurt the mobile experience, but instead of just objecting I built a quick version with a collapsible drawer so we could compare. We went with the drawer. I've learned that showing an alternative works far better than arguing about one."

"**Tight deadline?**" → Talk about scoping: shipping the core flow properly and flagging what you cut, rather than shipping everything half-done. Managers hire people who communicate trade-offs early.

"**Your weakness?**" → Real, but bounded, plus the fix:

"I used to over-polish UI details before the feature was fully working. I've started building the working version first and reviewing the polish at the end — it keeps me from spending an hour on a hover state that gets redesigned anyway." *(Never "I'm a perfectionist" or "I work too hard". Both read as rehearsed.)*

"**Where do you see yourself in 3 years?**" → "Owning a significant part of a frontend codebase — the person who's the reference point for performance and component architecture on the team." *(Aligns with 18 months. Don't say "starting my own company" or "moving abroad".)*

"**How do you work with designers?**" → "I go through the design early and ask about the states the mock doesn't show — loading, error, empty, long text, small screens. Catching those in a 10-minute conversation is far cheaper than in QA."

"**How do you keep code maintainable?**" → "Small components with one job, clear naming, no magic numbers, shared logic in custom hooks, and a folder structure someone new can navigate. My test is whether a teammate can change it without asking me how it works."

| 6. • Questions YOU ask them — *never say "no, I'm good"*

Have 4 ready; ask 2–3. Ending with no questions reads as no interest.

Technical (ask these in the tech round):

1. "Is the frontend on the App Router or Pages Router, and are you migrating?"
2. "How do you currently handle rendering strategy for the public pages — mostly SSG/ISR, or SSR?"
3. "Do you track Core Web Vitals in production, and is there a performance budget?"
4. "What does the frontend-to-backend workflow look like — do you get API contracts up front?"
5. "Is there a shared component library, or does each page build its own?"

Role / team (ask in the manager or HR round):

6. "What would a successful first three months in this role look like?"
7. "How big is the frontend team, and would I own features end to end?"
8. "Is there code review, and who reviews frontend work?"

The one that makes you memorable — ask it in the manager round:

"What's the biggest frontend problem you're hoping this hire helps solve?"

Then **listen**, and connect your answer to it. That's how you close.

| 7. Logistics for tomorrow

- Laptop charged + charger · backup mobile hotspot · water · pen and notebook.
- 2 printed copies of your resume, plus a copy open on your laptop.
- Your GitHub open in a tab — and **your best 2 projects running**, so "can you show me?" gets a yes in five seconds, not a five-minute build.
- Reach 15 minutes early. If it's online, join 5 minutes early and test camera/mic.
- Dress: smart casual — collared shirt. Slightly overdressed beats underdressed.
- Know your own resume cold. **Anything on it is fair game** — if you listed a technology you touched once, be ready to say "I've used it at a project level."

| 8. If you get stuck in the room

- **Don't freeze, narrate.** "Let me think about that for a second."

- **Don't bluff.** "I haven't used that directly. My understanding is X — is that the direction you mean?" That answer has never cost anyone an offer. Bluffing has.
- **Ask for the goal.** "Is the tricky part here the state, or the API side?"
- If you fumble a question, let it go. Candidates lose interviews by mentally re-litigating question 4 while answering question 7.

09 · Pitching Your Projects

Budget: 45 minutes — out loud, standing up, twice.

⚠ **Verify this file against your actual resume.** Your resume PDF is on your local machine and this session can't read it, so the project list below comes from your **public GitHub repositories**. The interviewer will be reading the PDF, so where the two disagree, the PDF wins — edit this file to match.

Why this matters more than any single technical question

At 4–6 LPA, everyone in the pipeline has a todo app. **The differentiator isn't the project — it's whether you can explain a decision you made inside it.** A candidate who says *"I used index as key at first and it broke when I deleted rows, so I switched to ids"* beats a candidate with a flashier project and no story, every time. Interviewers are listening for evidence you've hit real problems.

Your public repos — the ones worth talking about

Repo	What it demonstrates	Use it to answer
MultistepForm	multi-step state, validation, controlled inputs	"state management", "forms", "a hard bug"
medify	API integration, booking/search flow, real app	"REST APIs", "biggest project"
expenseTracker	CRUD, derived totals, list rendering	"state + derived data"
task-manager	CRUD, filtering, persistence	"component design"
xpagination	pagination logic, edge cases, API data	"pagination", "edge cases"
bot-ai	async flows, streaming/chat UI, loading states	"async", "loading states"
Portfolio / sujith-mondithoka.github.io	responsive layout, deployment, performance	"responsive design", "SEO/perf"
NFT-project / order-summary-component / XIntroSection	pixel-accurate design → code	"translating wireframes" ← <i>the JD's opening line</i>

Pick your top 2 and prepare them properly. Mention the rest only in passing. I'd lead with **medify** (most app-like) and **MultistepForm** (best decision story).

The 2-minute project pitch — the structure to fill in

Use this skeleton for each of your two main projects:

1. **What it is, in one line.** *"Medify is a doctor-appointment booking app — search hospitals by state and city, pick a slot, and it holds your bookings."*
2. **Your stack and why.** *"React with functional components and hooks, React Router for routes, and a public REST API for the hospital data. Styled with [X] because [reason]."*
3. **One hard problem and how you solved it.** ← **The part they're actually listening for.** *"The search results took a noticeable moment and the UI flashed empty, so I added explicit loading and empty states rather than rendering nothing — and I debounced the input so it wasn't firing a request per keystroke."*
4. **One thing you'd do differently now.** ← **The maturity signal.** *"I'd move the API calls out of the components into a `useHospitals` custom hook — right now the fetch logic is duplicated in two places. And I'd add error handling for a failed request, which I skipped."*

That fourth point is the single highest-leverage sentence in your interview.

Self-critique reads as senior. It also lets you control the follow-up: they'll ask about *the thing you already thought through*, not the thing you're weak on.

Prepared answers for the standard project questions

"Walk me through your biggest project." → the 4-part pitch above.

"Why did you choose React for this?"

"Component reuse and declarative state. The booking list, the card and the form all repeat across pages — writing them once and passing props saved a lot of duplication, and I don't have to manually sync the DOM when the data changes."

"How did you handle state?"

"`useState` locally, lifted to the nearest common parent when two components needed it. I didn't add Redux — for that size it'd be ceremony. If it grew, I'd reach for Context for the user/session and React Query for server data."

"How did you handle API errors?" → Be honest. If you didn't handle them fully:

"Only partially — I handled the loading state but not failures. Reading up on it since, I'd wrap the call in try/catch, check `res.ok` because fetch doesn't throw on a 404, and render a retry state." (*Honesty + the correct fix beats bluffing.*)

"How would you make it faster?" → Pull from file 04: image optimisation, code splitting the routes, debouncing the search, virtualising the long list, and

measuring with Lighthouse first.

"How would you add tests?"

"React Testing Library for components — testing what the user sees rather than internal state — and Jest for utility functions. I'd start with the form validation and the booking flow, since those are where a regression actually costs something."

"Did you deploy it?" → Say where (Netlify/Vercel/GitHub Pages) and mention anything you hit: environment variables, build config, routing 404s on refresh with client-side routing. Deployment problems are great, concrete, real-work stories.

• Tonight's homework — do not skip this

1. **Open your top 2 repos and re-read your own code.** Nothing sinks a candidate faster than not recognising code they wrote six months ago.
2. For each, write down: **one decision** you made and **one thing you'd change**.
3. Make sure both **run locally** — `npm install && npm run dev` — and have the deployed links ready to paste.
4. Say the 2-minute pitch out loud. Time it. If it's over 2:30, cut the setup.

Bridging thin experience honestly

If Next.js on your resume is tutorial-level, use this line:

"My production-style work is React; with Next.js I've built at a project level rather than shipped at scale. But I understand what it's solving — server rendering for SEO and first paint, ISR for content that changes periodically — and I'd be able to contribute quickly."

Then immediately demonstrate it using file 03. Interviewers respect a candidate who marks the boundary of their own knowledge accurately, because it means they can trust everything said inside the boundary. That trust is what gets you hired.

10 · Final Cheat Sheet — TOMORROW MORNING ONLY

07:20 - 08:00. Nothing else. No new material.

If a line here doesn't ring a bell, open that file for 3 minutes and come back.

| The 8 sentences that carry this interview

1. **Why Next.js:** "Plain React ships an empty HTML shell that the browser fills in — bad for first paint and unreliable for crawlers. Next renders on the server, so the browser and Google both get real HTML immediately."
 2. **Rendering strategies:** "SSG at build, SSR per request, ISR is SSG that regenerates on a timer, CSR in the browser. Bank listing pages → ISR. A student's own application page → SSR or CSR behind auth."
 3. **Server vs Client Components:** "Server by default, ships zero JS; push `'use client'` down to the interactive leaves."
 4. **Core Web Vitals:** "LCP under 2.5 s — is it there. CLS under 0.1 — does it stay still. INP under 200 ms — does it respond."
 5. **Closure:** "A function that remembers the scope it was created in. It's how `useState` and `debounce` both work."
 6. **Keys:** "Stable identity across renders. Index breaks the moment you delete or reorder — React reuses the wrong DOM node."
 7. **Four UI states:** "Loading, error, empty, success. Empty is the one people forget."
 8. **Business tie-in (say once):** "Since students find you through Google, server rendering and load speed here are lead generation, not polish."
-

| JavaScript

- `var` function-scoped · `let` / `const` block-scoped + TDZ · `const` ≠ immutable object
- Event loop: **microtasks (Promises) before macrotasks (setTimeout)** → 1 4 3 2
- `Promise.all` parallel · `allSettled` all results · `race` first settled · `any` first success
- Arrow functions take `this` from the enclosing scope and can't be rebound
- `??` only falls back on null/undefined; `||` falls back on any falsy (`0` !)
- Spread = **shallow** copy · deep = `structuredClone`
- `fetch` **does not throw on 404/500** — check `res.ok`
- Debounce = after they stop (search) · Throttle = at most once per N ms (scroll)
- `typeof null === 'object'` · `NaN !== NaN` · `Array.isArray()`

React

- Re-render causes: own state · props · **parent re-rendered** · context value
- Reduce: `React.memo` → `useCallback` / `useMemo` → move state down → `children`
- Never mutate state — new reference or no re-render (`Object.is`)
- `setCount(c => c + 1)` when the new value depends on the old
- `useEffect` = sync with the outside world; cleanup `return` ; don't use it for derived data
- `useMemo` value · `useCallback` function · `useRef` persists without re-rendering
- Hooks: top level only — React tracks them **by call order**
- Context re-renders every consumer; memoize the provider value
- `React.lazy` + `Suspense` = code splitting

Next.js

- `app/page.js` routing · `[id]` dynamic · `layout.js` · `loading.js` · `error.js`
- `fetch(url, { next: { revalidate: 60 } })` = ISR · `{ cache: 'no-store' }` = SSR
- `next/image` : WebP/AVIF, srcset, lazy, reserves space (CLS) · `priority` for the LCP image
- `metadata` / `generateMetadata` for per-page title & description
- Hydration = server HTML + client listeners; mismatches come from `Date` , `random` , `window`
- `<Link>` client-side nav + prefetch; `<a>` = full reload
- App Router → `next/navigation` ; Pages Router → `next/router`
- Only `NEXT_PUBLIC_*` reaches the browser — never a secret

CSS

- `box-sizing: border-box` · Flex = 1D, Grid = 2D
- `grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))` = responsive, no media queries
- Mobile-first with `min-width` · viewport meta tag or nothing works
- `display:grid; place-items:center` to centre
- Animate `transform` / `opacity` only — they skip layout and paint

APIs

- 401 = who are you · 403 = I know, still no · 429 = rate limited
- `AbortController` in the effect cleanup → no stale state, no race condition
- CORS is a browser rule, fixed on the **backend**

Machine coding — the 6 things they're grading

1. `key={item.id}` , never the index
2. Derived data computed during render, not stored in state + effect

3. Empty state handled ("No results for...")
 4. Loading **and** error states, ideally with a retry
 5. Cleanup — `clearTimeout` / `clearInterval` / `abort`
 6. **Talking while you type**
-

| Opening & closing lines

"**Tell me about yourself**" → Present → Path → **Why here** (file 08 §1). Under 2 min.

Your closing question → *"What's the biggest frontend problem you're hoping this hire helps solve?"*

Filter questions — answer instantly, no hesitation:

18-month commitment ✓ · WFO ✓ · 1st & 3rd Saturdays ✓ · 10–11 AM login ✓

Salary → *"I saw the 4–6 LPA band and I'm comfortable in it; I'd expect to be placed based on how the technical rounds go."* **Never bid below 4.**

| The last 5 minutes before you walk in

- Close every tab except your GitHub and your two running projects.
- You know more than you think you do. Nobody expects perfection at this level — they expect someone who reasons clearly and doesn't bluff.
- **If you don't know something: say so, then reason out loud.** That answer has never lost anyone an offer. Bluffing has.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.